

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 3

За седьмой семестр

Тема: «Многослойный персептрон для XOR с ReLU на скрытых слоях: сравнение
активационных функций»

Выполнил:
Студент 4 курса
Группы ИИ-26
Згера Е. А.
Проверил:
Никитин С.К.

Вариант 5

Цель: На основе многослойного персептрона архитектуры 2-2-1 из ЛР №2, исследовать влияние активационной функции ReLU (Rectified Linear Unit) на скрытых слоях. Провести сравнительный анализ обучения сети с функцией активации Сигмоида (базовая конфигурация) и ReLU (новая конфигурация) по скорости сходимости, качеству обучения, форме разделяющей поверхности и устойчивости к инициализации весов.

Задачи лабораторной работы

1. Взять за основу код многослойного персептрона 2-2-1 с функцией потерь BCE из ЛР №2.
2. Реализовать активационную функцию ReLU на скрытом слое и осуществить корректное обратное распространение ошибки для ReLU. На выходном слое оставить Сигмоиду.
3. Обучить сеть на полной таблице истинности в двух конфигурациях:
 - **Конфигурация А (базовая):** MLP 2-2-1, Сигмоида на всех слоях, BCE, нормализованные целевые значения (0/1);
 - **Конфигурация В (новая):** MLP 2-2-1, ReLU на скрытом слое, Сигмоида на выходном, BCE, нормализованные (0/1).
4. Критерий остановки для обеих конфигураций: суммарная ошибка $E_s \leq E_e$ (как в ЛР №2).
5. Для каждой конфигурации зафиксировать:
 - Количество эпох до критерия остановки.
 - Итоговую суммарную ошибку и ассигасу.
 - Среднюю абсолютную ошибку в исходной шкале $[c_0; c_1]$.
6. Провести не менее 5 запусков обучения для каждой конфигурации с разными seed и сравнить устойчивость.
7. Построить графики:
 - График сходимости (суммарная ошибка E_s от эпохи) для обеих конфигураций на одних осях.
 - Диаграмма разброса числа эпох по 5 запускам (столбики) для конфигурации А и В.
8. Визуализация разделяющей поверхности:
 - Для представительного экземпляра каждой конфигурации построить карту (heatmap) выходного значения сети на плоскости $(A, B) \in [-10; 10]^2$ с наложением 4 точек обучающей выборки.
 - Описать, как ReLU изменяет форму разделяющей поверхности (кусочно-линейные границы vs гладкие).
9. Реализовать режим функционирования сети:
 - Пользователь вводит (A, B) из $[-10; 10]$.
 - Сеть выводит: выход в нормализованной шкале, выход в исходной шкале $[c_0; c_1]$, класс.
 - Продемонстрировать на 4 обучающих примерах + 2–3 дополнительных парах.
10. Создать таблицу сравнения (статистика по 5 запускам, MAE для обеих конфигураций).
11. Вывод: ответить на все вопросы и провести анализ результатов.

Вариант 5 ($c_0 = 1, c_1 = 8$)

A	B	$A \oplus B$
1	1	1
1	8	8
8	1	8
8	8	1

Описание изменений в архитектуре и алгоритме обучения:

Архитектура сети не изменилась: MLP 2-2-1 (два входных нейрона, два скрытых, один выходной), сигмоида на обоих слоях.

Код программы

```
# -*- coding: utf-8 -*-

import numpy as np
import matplotlib.pyplot as plt

np.set_printoptions(precision=4, suppress=True)

c0, c1 = 1, 8

X = np.array([[c0, c0], [c0, c1], [c1, c0], [c1, c1]], dtype=float)
y_raw = np.array([[c0], [c1], [c1], [c0]], dtype=float)
y_bin = np.array([[0.], [1.], [1.], [0.]])

LR = 0.1
MAX_EPOCHS = 20000
EE_BCE = 0.35
SEEDS = [0, 1, 2, 3, 4]

CFG_NAMES = {"A": "Конфигурация A (Сигмоида)", "V": "Конфигурация B (ReLU)"}
CFG_COLORS = {"A": "tab:blue", "V": "tab:orange"}

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-np.clip(x, -500, 500)))

def relu(x):
    return np.maximum(0.0, x)

def hidden_activation(z, config):
    return sigmoid(z) if config == "A" else relu(z)

def hidden_derivative(z, h, config):
    if config == "A":
        return h * (1.0 - h)
    return (z > 0).astype(float)

def init_weights(seed, config="A", n_in=2, n_hidden=2, n_out=1):
    rng = np.random.RandomState(seed)
    if config == "V":
        W1 = rng.uniform(-1.0, 1.0, (n_in, n_hidden))
        b1 = rng.uniform(-5.0, 5.0, (1, n_hidden))
        W2 = rng.uniform(-1.0, 1.0, (n_hidden, n_out))
    else:
        W1 = rng.uniform(-0.5, 0.5, (n_in, n_hidden))
        b1 = np.zeros((1, n_hidden))
        W2 = rng.uniform(-0.5, 0.5, (n_hidden, n_out))
        b2 = np.zeros((1, n_out))
    return W1, b1, W2, b2

def forward(Xb, W1, b1, W2, b2, config):
```

```

z1 = Xb.dot(W1) + b1
h_out = hidden_activation(z1, config)
o_out = sigmoid(h_out.dot(W2) + b2)
return z1, h_out, o_out

def bce_loss(o_out, y, eps=1e-9):
    o_clip = np.clip(o_out, eps, 1 - eps)
    return -np.mean(y * np.log(o_clip) + (1 - y) * np.log(1 - o_clip))

def train(seed, config, lr=LR, max_epochs=MAX_EPOCHS, Ee=EE_BCE):
    W1, b1, W2, b2 = init_weights(seed, config)
    errors = []
    converged_epoch = None
    n = X.shape[0]

    for epoch in range(max_epochs):
        z1, h_out, o_out = forward(X, W1, b1, W2, b2, config)

        Es = bce_loss(o_out, y_bin)
        errors.append(Es)

        if Es <= Ee:
            converged_epoch = epoch
            break

        d_output = (o_out - y_bin) / n
        d_hidden = d_output.dot(W2.T) * hidden_derivative(z1, h_out, config)

        W2 -= h_out.T.dot(d_output) * lr
        b2 -= np.sum(d_output, axis=0, keepdims=True) * lr
        W1 -= X.T.dot(d_hidden) * lr
        b1 -= np.sum(d_hidden, axis=0, keepdims=True) * lr

    return dict(W1=W1, b1=b1, W2=W2, b2=b2, errors=errors,
                converged_epoch=converged_epoch, config=config)

def nearest_class(value, c0, c1):
    return c0 if abs(value - c0) <= abs(value - c1) else c1

def rescale_output(o_out, c0, c1):
    return c0 + o_out * (c1 - c0)

def degenerate_neurons(result):
    z1, _, _ = forward(X, result["W1"], result["b1"], result["W2"], result["b2"], result["config"])
    dead = int(np.sum(np.all(z1 <= 0, axis=0)))
    linear = int(np.sum(np.all(z1 > 0, axis=0)))
    return dead, linear

def evaluate_run(result):
    _, _, o_out = forward(X, result["W1"], result["b1"], result["W2"], result["b2"], result["config"])
    pred_scale = rescale_output(o_out.flatten(), c0, c1)
    true_scale = y_raw.flatten()
    pred_class = np.array([nearest_class(v, c0, c1) for v in pred_scale])
    accuracy = np.mean(pred_class == true_scale)
    mae = np.mean(np.abs(pred_scale - true_scale))
    Es_final = bce_loss(o_out, y_bin)
    return accuracy, mae, Es_final

def run_experiments():
    all_results = {"A": [], "V": []}
    for config in ["A", "V"]:
        print(f"\n{CFG_NAMES[config].upper()}: BCE, Ee = {EE_BCE}")
        for seed in SEEDS:
            res = train(seed, config)
            acc, mae, Es_final = evaluate_run(res)
            res.update(seed=seed, accuracy=acc, mae=mae, Es_final=Es_final)
            all_results[config].append(res)
            status = f"эпоха {res['converged_epoch']}" if res["converged_epoch"] is not None \
                else f"НЕ СОШЛАСЬ за {MAX_EPOCHS} эпох"
            extra = ""
            if config == "V":
                dead, lin = degenerate_neurons(res)
                extra = f", мёртвых={dead}, линейных={lin}"
            print(f"seed={seed}: {status}, Es_final={Es_final:.4f}, "
                  f"accuracy={acc:.2f}, MAE={mae:.2f}{extra}")
    return all_results

def comparison_table(all_results):
    header = f"{'Метрика':<34}{'A (Сигмоида)':>16}{'B (ReLU)':>16}"
    print("\nТАБЛИЦА СРАВНЕНИЯ (статистика по 5 запускам)")
    print(header)

```

```

print("-" * len(header))

def stats(results):
    conv = [r for r in results if r["converged_epoch"] is not None]
    ep = [r["converged_epoch"] for r in conv]
    return {
        "Сошлось запусков": f"{len(conv)}/{len(results)}",
        "Эпох, min": f"{min(ep)}" if ep else "-",
        "Эпох, max": f"{max(ep)}" if ep else "-",
        "Эпох, среднее": f"{np.mean(ep):.1f}" if ep else "-",
        "Эпох, std": f"{np.std(ep):.1f}" if ep else "-",
        "Итоговая Es (BCE), среднее": f"{np.mean([r['Es_final'] for r in results]):.4f}",
        "Аккурату, среднее": f"{np.mean([r['accuracy'] for r in results]):.2f}",
        "MAE в шкале [c0;c1], среднее": f"{np.mean([r['mae'] for r in results]):.3f}",
        "MAE в шкале [c0;c1], std": f"{np.std([r['mae'] for r in results]):.3f}",
    }

sA, sV = stats(all_results["A"]), stats(all_results["V"])
for key in sA:
    print(f"{key:<34}{sA[key]:>16}{sV[key]:>16}")

def plot_convergence(all_results, rep_seed=0):
    fig, ax = plt.subplots(figsize=(8, 5))
    for config in ["A", "V"]:
        rep = next(r for r in all_results[config] if r["seed"] == rep_seed)
        ax.plot(rep["errors"], color=CFG_COLORS[config], label=CFG_NAMES[config])
    ax.axhline(EE_BCE, color="gray", linestyle="--", alpha=0.7, label=f"Ee={EE_BCE}")
    ax.set_xlabel("Эпоха")
    ax.set_ylabel("Es (BCE)")
    ax.set_title(f"Сходимость: Сигмоида vs ReLU (seed={rep_seed})")
    ax.legend()
    fig.tight_layout()
    plt.savefig("plot_1_convergence.png", dpi=150)
    plt.show()

def plot_epochs_bar(all_results):
    fig, ax = plt.subplots(figsize=(9, 5))
    x = np.arange(len(SEEDS))
    width = 0.35

    for i, config in enumerate(["A", "V"]):
        epochs = [r["converged_epoch"] if r["converged_epoch"] is not None else MAX_EPOCHS
                  for r in all_results[config]]
        bars = ax.bar(x + (i - 0.5) * width, epochs, width,
                     label=CFG_NAMES[config], color=CFG_COLORS[config])
        for bar, r in zip(bars, all_results[config]):
            if r["converged_epoch"] is None:
                bar.set_hatch("/")
                bar.set_edgecolor("red")

    ax.set_xticks(x)
    ax.set_xticklabels([f"seed={s}" for s in SEEDS])
    ax.set_ylabel("Эпох до сходимости (MAX_EPOCHS, если не сошлось)")
    ax.set_title("Разброс числа эпох по 5 запускам (штриховка = не сошлось)")
    ax.legend()
    fig.tight_layout()
    plt.savefig("plot_2_epochs_bar.png", dpi=150)
    plt.show()

def plot_decision_surfaces(all_results, rep_seed=0):
    a_range = np.linspace(-10, 10, 200)
    b_range = np.linspace(-10, 10, 200)
    AA, BB = np.meshgrid(a_range, b_range)
    grid = np.column_stack([AA.ravel(), BB.ravel()])

    fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))
    for ax, config in zip(axes, ["A", "V"]):
        rep = next(r for r in all_results[config] if r["seed"] == rep_seed)
        _, _, o = forward(grid, rep["w1"], rep["b1"], rep["w2"], rep["b2"], config)
        ZZ = o.reshape(AA.shape)
        cs = ax.contourf(AA, BB, ZZ, levels=20, cmap="coolwarm", vmin=0, vmax=1)
        fig.colorbar(cs, ax=ax, label="ŷ (нормализованный выход, 0..1)")
        if ZZ.min() < 0.5 < ZZ.max():
            ax.contour(AA, BB, ZZ, levels=[0.5], colors="black", linewidths=2)
            ax.scatter(X[:, 0], X[:, 1], c=y_bin.flatten(), cmap="coolwarm",
                      edgecolors="black", s=120, linewidths=1.5)
        ax.set_title(CFG_NAMES[config])
        ax.set_xlabel("A")
        ax.set_ylabel("B")

    fig.suptitle(f"Разделяющая поверхность (A,B ∈ [-10;10], seed={rep_seed})")

```

```

fig.tight_layout()
plt.savefig("plot_3_decision_surfaces.png", dpi=150)
plt.show()

def plot_mae_comparison(all_results):
    maes = [np.mean([r["mae"] for r in all_results[c]] for c in ["A", "V"])
    fig, ax = plt.subplots(figsize=(6, 5))
    bars = ax.bar([CFG_NAMES["A"], CFG_NAMES["V"]], maes,
                  color=[CFG_COLORS["A"], CFG_COLORS["V"]])
    for bar, val in zip(bars, maes):
        ax.text(bar.get_x() + bar.get_width() / 2, val, f"{val:.2f}", ha="center", va="bottom")
    ax.set_ylabel(f"Средняя MAE в шкале [{c0}; {c1}] (по 5 запускам)")
    ax.set_title("Точность восстановления исходной шкалы")
    plt.setp(ax.get_xticklabels(), fontsize=8)
    fig.tight_layout()
    plt.savefig("plot_4_mae_comparison.png", dpi=150)
    plt.show()

def predict(a, b, result):
    _, _, o_out = forward(np.array([[a, b]], dtype=float),
                          result["w1"], result["b1"], result["w2"], result["b2"], result["config"])
    y_hat_norm = o_out.item()
    y_hat_scaled = rescale_output(y_hat_norm, c0, c1)
    y_class = nearest_class(y_hat_scaled, c0, c1)
    return y_hat_norm, y_hat_scaled, y_class

def demo_predictions(result):
    print(f"\nРЕЖИМ ФУНКЦИОНИРОВАНИЯ СЕТИ ({CFG_NAMES[result['config']]}, seed={result['seed']})")

    print("\n--- Все 4 примера обучающей таблицы ---")
    for a, b in X:
        n_, s_, c_ = predict(a, b, result)
        print(f"Вход: ({a:.0f}, {b:.0f}) ->  $\hat{y}_{norm}={n_:.4f}$ ,  $\hat{y}_{scaled}={s_:.2f}$ , класс≈{c_}")

    print("\n--- Дополнительные пары (вне обучающей выборки) ---")
    for a, b in [(3, 3), (2, 7), (5, 5), (-4, 6)]:
        n_, s_, c_ = predict(a, b, result)
        print(f"Вход: ({a}, {b}) ->  $\hat{y}_{norm}={n_:.4f}$ ,  $\hat{y}_{scaled}={s_:.2f}$ , класс≈{c_}")

def interactive_mode(result):
    print("\n--- Интерактивный режим (введите 'q' для выхода) ---")
    while True:
        raw = input("Введите A,B через пробел или запятую в диапазоне [-10;10]: ").strip()
        if raw.lower() == "q":
            break
        try:
            a_str, b_str = raw.replace(",", " ").split()
            a, b = float(a_str), float(b_str)
        except ValueError:
            print("Не удалось разобрать ввод, попробуйте снова (пример: 3 7).")
            continue
        if not (-10 <= a <= 10 and -10 <= b <= 10):
            print("Значения должны быть в диапазоне [-10;10].")
            continue
        n_, s_, c_ = predict(a, b, result)
        print(f" $\hat{y}_{norm}={n_:.4f}$  |  $\hat{y}$  в шкале [{c0};{c1}]= ${s_:.2f}$  | ближе к классу {c_}")

if __name__ == "__main__":
    all_results = run_experiments()
    comparison_table(all_results)

    plot_convergence(all_results, rep_seed=0)
    plot_epochs_bar(all_results)
    plot_decision_surfaces(all_results, rep_seed=0)
    plot_mae_comparison(all_results)

    rep_V = next((r for r in all_results["V"] if r["converged_epoch"] is not None),
                  all_results["V"][0])
    demo_predictions(rep_V)

```

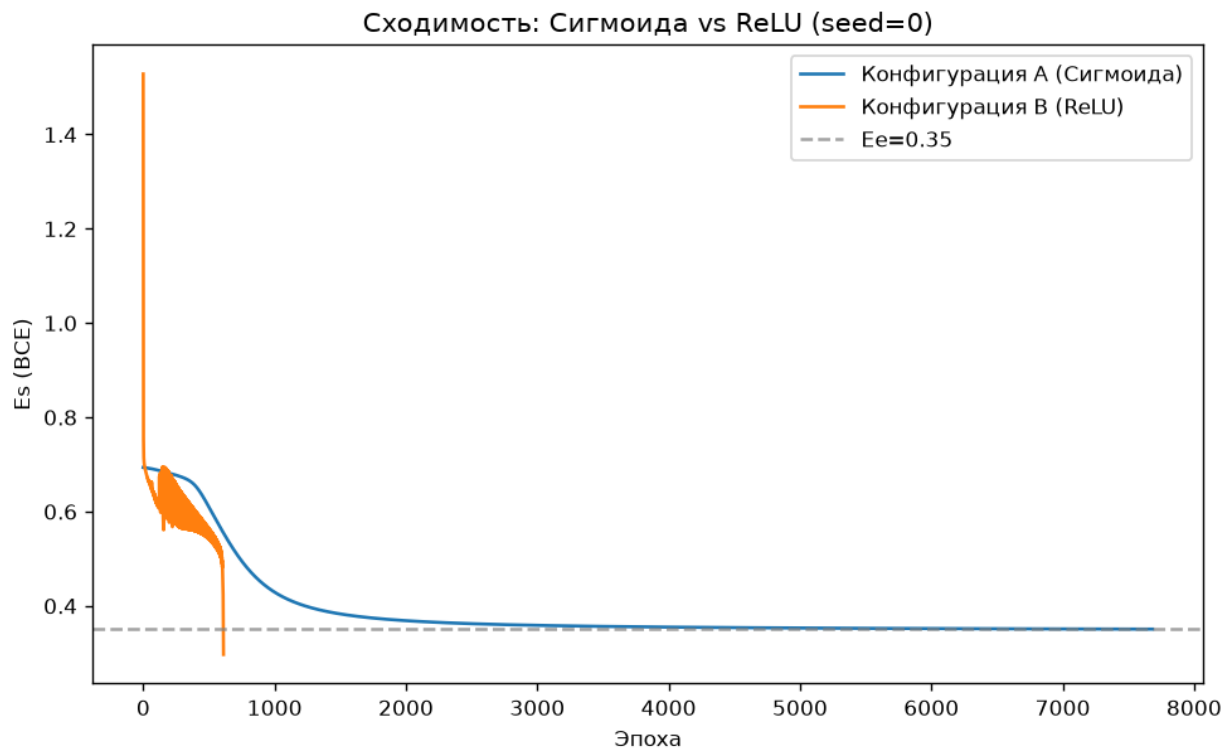
Таблица результатов:

КОНФИГУРАЦИЯ А (СИГМОИДА): BCE, Ee = 0.35
seed=0: эпоха 7684, Es_final=0.3500, accuracy=0.75, MAE=1.77
seed=1: эпоха 10340, Es_final=0.3500, accuracy=0.75, MAE=1.77
seed=2: эпоха 7793, Es_final=0.3500, accuracy=0.75, MAE=1.77
seed=3: эпоха 10721, Es_final=0.3500, accuracy=0.75, MAE=1.77
seed=4: эпоха 7490, Es_final=0.3500, accuracy=0.75, MAE=1.77

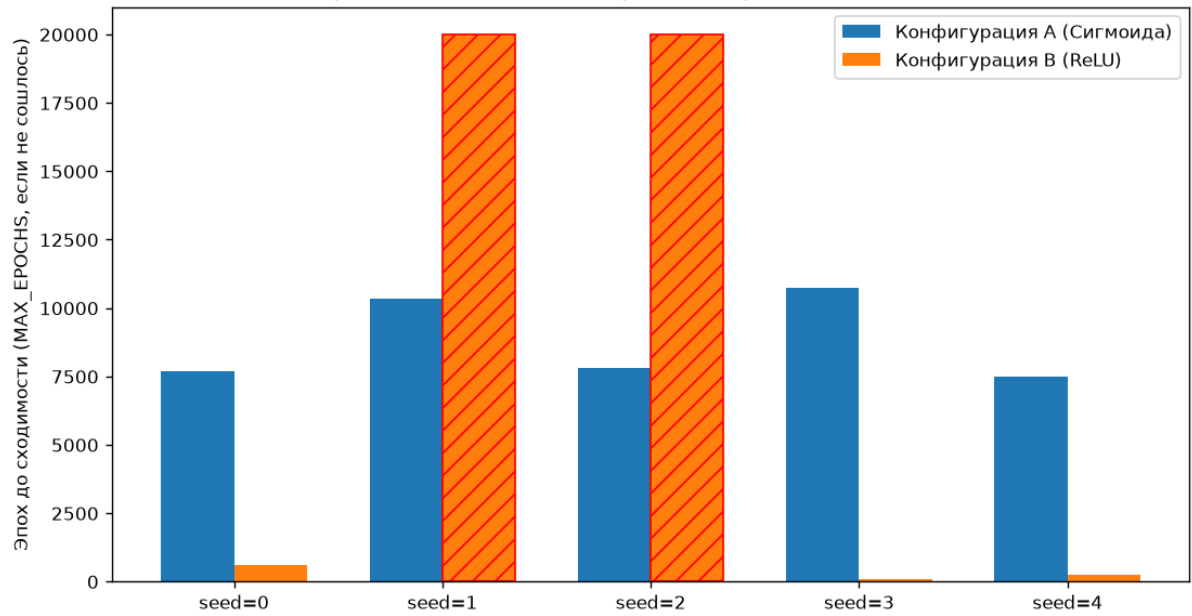
КОНФИГУРАЦИЯ В (ReLU): BCE, Ee = 0.35
seed=0: эпоха 611, Es_final=0.2962, accuracy=1.00, MAE=1.68, мёртвых=0, линейных=1
seed=1: НЕ СОШЛАСЬ за 20000 эпох, Es_final=0.6931, accuracy=0.50, MAE=3.50, мёртвых=2, линейных=0
seed=2: НЕ СОШЛАСЬ за 20000 эпох, Es_final=0.6931, accuracy=0.50, MAE=3.50, мёртвых=2, линейных=0
seed=3: эпоха 95, Es_final=0.3495, accuracy=0.75, MAE=1.92, мёртвых=0, линейных=1
seed=4: эпоха 244, Es_final=0.3391, accuracy=0.75, MAE=1.85, мёртвых=0, линейных=1

ТАБЛИЦА СРАВНЕНИЯ (статистика по 5 запускам)		
Метрика	А (Сигмоида)	В (ReLU)
Сошлось запусков	5/5	3/5
Эпох, min	7490	95
Эпох, max	10721	611
Эпох, среднее	8805.6	316.7
Эпох, std	1416.8	216.8
Итоговая Es (BCE), среднее	0.3500	0.4742
Ассигасу, среднее	0.75	0.70
MAE в шкале [с0;с1], среднее	1.773	2.490
MAE в шкале [с0;с1], std	0.000	0.828

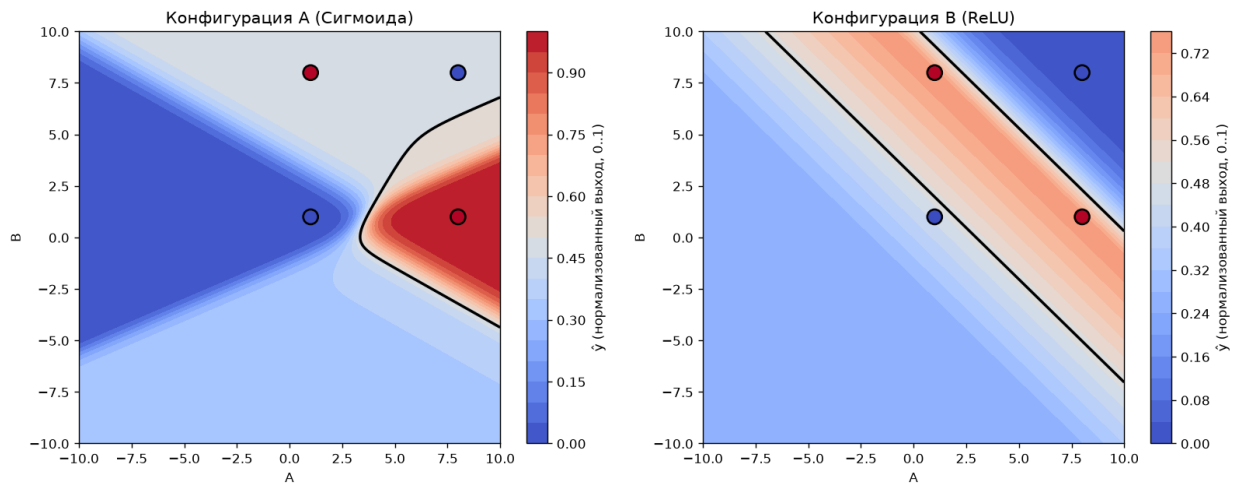
Блок визуализации:



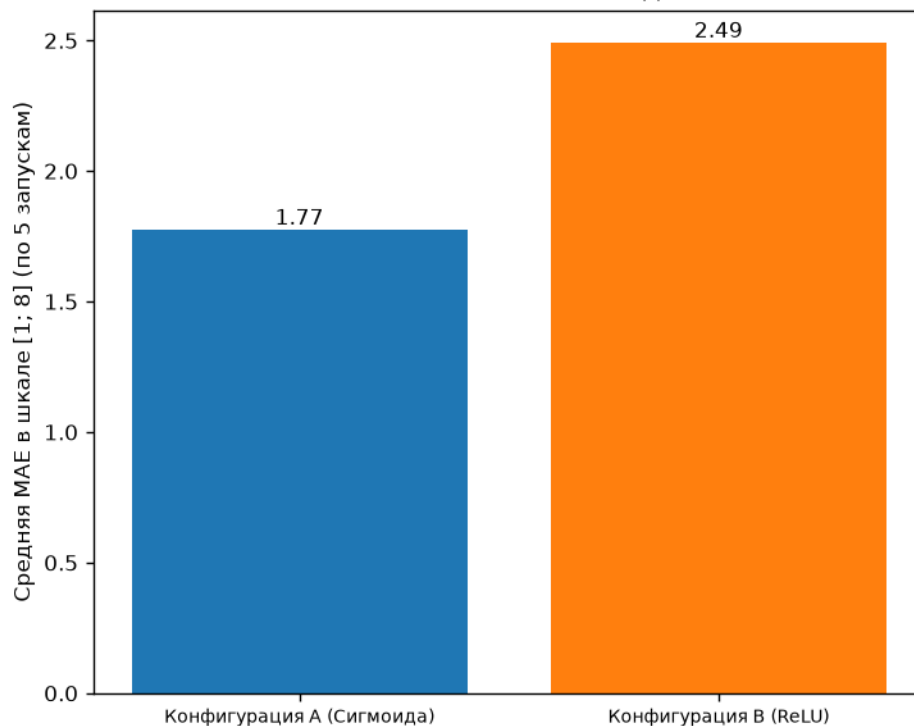
Разброс числа эпох по 5 запускам (штриховка = не сошлось)



Разделяющая поверхность (A,B ∈ [-10;10], seed=0)



Точность восстановления исходной шкалы



Результаты работы обученной сети в режиме функционирования:

РЕЖИМ ФУНКЦИОНИРОВАНИЯ СЕТИ (Конфигурация В (ReLU), seed=0)

--- Все 4 примера обучающей таблицы ---

Вход: (1, 1) -> $\hat{y}_{\text{norm}}=0.4534$, $\hat{y}_{\text{scaled}}=4.17$, класс≈1

Вход: (1, 8) -> $\hat{y}_{\text{norm}}=0.7510$, $\hat{y}_{\text{scaled}}=6.26$, класс≈8

Вход: (8, 1) -> $\hat{y}_{\text{norm}}=0.7510$, $\hat{y}_{\text{scaled}}=6.26$, класс≈8

Вход: (8, 8) -> $\hat{y}_{\text{norm}}=0.0080$, $\hat{y}_{\text{scaled}}=1.06$, класс≈1

--- Дополнительные пары (вне обучающей выборки) ---

Вход: (3, 3) -> $\hat{y}_{\text{norm}}=0.6424$, $\hat{y}_{\text{scaled}}=5.50$, класс≈8

Вход: (2, 7) -> $\hat{y}_{\text{norm}}=0.7510$, $\hat{y}_{\text{scaled}}=6.26$, класс≈8

Вход: (5, 5) -> $\hat{y}_{\text{norm}}=0.5640$, $\hat{y}_{\text{scaled}}=4.95$, класс≈8

Вход: (-4, 6) -> $\hat{y}_{\text{norm}}=0.4534$, $\hat{y}_{\text{scaled}}=4.17$, класс≈1

Вывод:

1. Какая функция обеспечила более быструю сходимость? Более устойчива?

Быстрее сходится ReLU: в среднем 317 эпох (от 95 до 611) против 8806 эпох (от 7490 до 10721) у сигмоиды. Устойчивее сигмоида: сошлись все 5 запусков из 5, у ReLU только 3 из 5.

2. Как отличается форма разделяющей поверхности? Наблюдаются ли кусочно-линейные границы для ReLU?

У сигмоиды поверхность гладкая, выход плавно переходит от 0 к 1. У ReLU граница кусочно-линейная: состоит из прямых участков с изломами там, где нейрон переключается из «выключен» в «включён». Вне области данных скрытый слой растёт линейно, а выходная сигмоида насыщается, поэтому появляются широкие плоские зоны.

3. Почему ReLU часто предпочитают в глубоких сетях? Подтверждается ли это на XOR?

У ReLU нет насыщения при $z > 0$ (градиент равен 1), поэтому в глубоких сетях нет затухания градиента, а вычисления дешёвые. На XOR подтвердилась только скорость обучения. Преимущество по устойчивости не подтвердилось: из-за мёртвых нейронов ReLU сошёл хуже сигмоиды.

4. Изменилась ли способность решать XOR при переходе на ReLU?

Нет. XOR решается любой сетью с одним нелинейным скрытым слоем из двух нейронов, и ReLU тоже может его представить (в одном из запусков accuracy = 1.00). Неудачи отдельных запусков вызваны неудачной инициализацией, а не недостатком архитектуры.

5. Есть ли «мёртвые» нейроны? Как это повлияло на обучение?

Да. В двух запусках из пяти оба скрытых нейрона оказались мёртвыми ($z \leq 0$ на всех обучающих точках). Градиент по весам скрытого слоя стал нулевым, сеть выдавала константу 0,5, ошибка осталась на уровне $\ln 2 \approx 0,693$, accuracy = 0,50. В остальных запусках один нейрон был активен на всех точках и работал как линейный, поэтому нелинейность обеспечивал только второй. При стандартной инициализации из ЛР №2 (малые веса, нулевые смещения) ReLU не сошёл ни в одном запуске (0/5).

6. Как выбор активации соотносится с необходимостью многослойной архитектуры для XOR?

Для XOR нужны одновременно скрытый слой и нелинейность в нём. Без скрытого слоя

граница всегда линейна при любой активации. Если ReLU-нейроны работают в линейном режиме (излом вне области данных), сеть вырождается в логистическую регрессию и застревает на $E_s = \ln 2$. Активация определяет, как именно создаётся нелинейность (гладко или изломами), но необходимость скрытого слоя от неё не зависит.

7. Что показывает визуализация поверхности? Совпадает ли граница с ожидаемой для XOR?

Ожидаемая граница отделяет диагональную пару (1,1), (8,8) от антидиагональной (1,8), (8,1). На обучающих точках обе конфигурации расставляют классы верно, но при мягком критерии остановки $E_e = 0,35$ часть точек остаётся близко к порогу 0,5. Вне обучающей выборки у ReLU совпадение неполное из-за линейной экстраполяции и раннего останова. Чтобы поверхность точнее соответствовала XOR, нужно снизить E_e (например, до 0,1).

Анализ результатов: ReLU заметно быстрее, но менее надёжен: его успех зависит от того, попадут ли изломы в область данных. На XOR при таком малом размере сети преимущество ReLU сводится к скорости, а не к качеству и не к устойчивости.